

AWS Well-Architected Framework Review for ROCA Scripture & Lectionary Management Flow (Batch 4 of 4) — Comprehensive Pillar Assessment and Cross-Batch Remediation Roadmap

1 Executive Summary

This final batch of ROCA's AWS Well-Architected Review reveals that Flow 8 — Scripture & Lectionary Management — introduces seven new risks atop six systemic architectural gaps that have compounded unremediated since Batch 1, bringing the platform's total to 25 distinct findings (7 P0-Critical, 9 P1-High, 9 P2-Medium) across all eight flows. The most urgent new risk is the absence of a circuit breaker on the ESV API's 5,000 requests/day hard limit [1, 2]: without graceful degradation through the Bible.api and local KJV SQLite fallback tiers, a single high-traffic Sunday morning could leave congregations without scripture text. This joins the platform-wide absence of DLQs, WAF/GuardDuty, and JWT rotation [3, 4] — gaps that now span 29 additional endpoints handling sensitive spiritual data including bookmarks, annotations, and reading plan progress.

Two architectural decisions demand near-term correction. First, the last-write-wins offline sync strategy [5] risks silently overwriting user-generated devotional content across devices; migrating to AWS AppSync's Optimistic Concurrency or Amplify DataStore's delta synchronization [6, 7] would preserve data integrity. Second, reading plan streak tracking with its 48-hour grace period [5] lacks conditional writes, exposing it to race conditions on concurrent multi-device completions — DynamoDB conditional expressions checking last-completion timestamps would enforce atomicity [8, 9].

The consolidated 90-day remediation roadmap requires approximately 95–115 engineering days across three tiers. The P0-Critical tier (Days 1–30) should prioritize the Bible API circuit breaker, DLQ deployment, and WAF/GuardDuty activation. The P1-High tier (Days 31–60) addresses Step Functions migration for embedded state machines across all flows, AppSync offline sync, and PostgreSQL tsvector column optimization to sustain sub-300ms search at scale [10]. The P2-Medium tier (Days 61–90) captures cost wins — Polly audio pre-generation to eliminate redundant synthesis at \$16/million characters [11] and a Redis-to-Valkey migration for up to 60% caching cost reduction [12]. Organizations should treat the P0 tier as non-negotiable; the compounding of foundational security and reliability gaps across all eight ROCA flows represents risk that grows with every unaddressed sprint.

2 Introduction and Flow 8 Architecture Overview

This report constitutes the fourth and final batch of the AWS Well-Architected Framework Review for ROCA, evaluating Flow 8 — Scripture & Lectionary Management — the spiritual backbone that connects the liturgical calendar, events, devotionals, and notifications to the church year. With this batch, all eight ROCA flows have now been assessed, enabling a consolidated view of architectural risk across the entire platform.

Flow 8 is among the most complex subsystems in ROCA, comprising 29 API endpoints organized across lectionary configuration, sermon series management, scripture delivery and search, reading plans, bookmarks and highlights, devotional generation, preaching history, and export [1]. The data layer consists of 8 PostgreSQL tables — `lectionary_config`, `lectionary_readings`, `scripture_assignments`, `sermon_series`, `reading_plans`, `reading_plan_progress`, `scripture_bookmarks`, and `preaching_history` — featuring UUID primary keys, foreign key relationships to organizations, users, and events, JSONB columns for flexible data structures, and TIMESTAMPTZ audit columns [1].

The system supports five distinct lectionary systems: the Revised Common Lectionary (RCL) for Presbyterian, Methodist, Lutheran, Anglican, and UCC traditions; the Roman Catholic lectionary with its additional weekday 2-year I/II cycle; the Book of Common Prayer for Episcopal and Anglican churches; a Custom system allowing user-defined cycles stored as JSONB; and a Freeform/None option for non-liturgical congregations [1]. Cycle year navigation follows the 3-year A/B/C pattern, advancing at Advent Sunday using the formula $[(\text{effectiveYear} - 2024) \% 3]$ [1]. The Easter Computus algorithm implements the Anonymous Gregorian Method, computing Easter dates through 13 intermediate variables and deriving all moveable feasts as offsets — Ash Wednesday at -46 days, Palm Sunday at -7, Ascension at +39, Pentecost at +49, Trinity Sunday at +56, and Corpus Christi at +60 [1].

Scripture text delivery relies on a tiered external API strategy: the ESV API serves as the primary source with a 5,000 requests/day limit, Bible.api provides additional translations, and a local SQLite database with public domain KJV text acts as the ultimate fallback, supporting 7 translations in total (ESV, NIV, NRSV, KJV, NLT, NAB, MSG) [1]. Server-side caching uses Redis with a 24-hour TTL per verse [1], while HTTP-level caching applies differentiated strategies — calendar data at max-age=3600 with stale-while-revalidate, cycle year data at max-age=86400, and sermon series with no-cache plus ETag-based conditional GETs [13]. Audio playback employs a two-tier architecture: pre-recorded HLS streams hosted on S3 with CloudFront CDN for ESV, NIV, and KJV at AAC 64kbps, and AWS

Polly Neural voice “Matthew” (EN-US) as a fallback for NRSV, NLT, NAB, and MSG translations, cached in S3 with a 30-day TTL [5]. Full-text search leverages PostgreSQL tsvector indexing across book name, chapter, and verse text for all translations, targeting sub-300ms response times at p95 [1].

The offline architecture uses SQLite on mobile with a 50MB on-device budget and 7-day pre-cache window — approximately 16.8MB allocated to audio, leaving roughly 33MB for text, metadata, highlights, and bookmarks [5]. Web offline support uses IndexedDB via Dexie.js, caching the last 7 days of lectionary readings with auto-sync on reconnect [13]. The 16 frontend components span 4 web pages, 3 mobile screens, 3 widgets, 4 shared/platform components, and 2 modals across React/TypeScript (web) and React Native (mobile) [1].

3 Pillar 1: Operational Excellence

Embedded State Machines and Missing Orchestration. The `scripture_assignments` table implements a four-state lifecycle (SUGGESTED → ASSIGNED → PUBLISHED → DELIVERED) [1], but like the onboarding 10-state, volunteer 8-state, and notification 6-state lifecycles identified in prior batches, this state machine is embedded in application code rather than orchestrated through AWS Step Functions [3]. This pattern makes state transitions opaque to operational tooling, complicates debugging, and prevents automated rollback on partial failures. The absence of EventBridge as a central event bus means that lectionary-to-calendar and lectionary-to-notification events remain tightly coupled through point-to-point integrations, a systemic gap that has persisted since Batch 1 [3].

Observability Gaps. Flow 8 introduces significant third-party API dependencies (ESV API, Bible.api) and a multi-tier audio fallback path (HLS → Polly), yet no X-Ray distributed tracing is in place to monitor latency across these external call chains [3]. Without tracing, operators cannot distinguish between a slow ESV API response, a Redis cache miss, and a Polly synthesis delay — all of which present differently to end users but require different remediation actions. The 29 API endpoints lack structured operational runbooks for common failure scenarios such as Bible API rate limit exhaustion, Polly service degradation, or tsvector index corruption.

Lectionary Data Accuracy. Supporting five distinct lectionary systems across multiple denominations introduces a data accuracy challenge that has no automated validation mechanism. There is no reconciliation process to verify that lectionary readings loaded for RCL, Roman Catholic, or BCP systems match authoritative published lectionaries for a given cycle year. Given that the Roman Catholic system adds a weekday 2-year I/II cycle on top of the standard 3-year Sunday cycle [1], the combinatorial complexity of readings that must be validated is substantial. An operational excellence best practice would be to implement automated regression tests against published lectionary tables for each supported system.

4 Pillar 2: Security

Systemic Authentication and Perimeter Gaps. All 29 scripture and lectionary endpoints inherit the platform-wide security deficiencies identified in Batch 1 and escalated through subsequent batches: JWT tokens lack rotation, PKCE, and refresh token reuse detection; WAF is not deployed to protect against API abuse; and GuardDuty is absent for threat detection [3, 4]. These gaps were originally rated P0-Critical and have compounded in severity as each new flow expands the unprotected API surface. With Flow 8, the attack surface now includes endpoints that accept JSONB payloads for custom lectionary configurations and freeform scripture references, creating injection vectors if input validation is insufficient.

API Key Management. The ESV API and Bible.api integrations require API keys for authentication [1], but the current architecture does not indicate use of AWS Secrets Manager for secure storage and automatic rotation. Storing API keys in Lambda environment variables — a pattern already flagged for Stripe API keys in Batch 2’s PCI DSS findings [14] — exposes credentials to anyone with Lambda console access and prevents automated rotation without redeployment.

Data Protection. Scripture bookmarks and annotations (stored in `scripture_bookmarks` with BOOKMARK, HIGHLIGHT, and ANNOTATION types [1]) contain user-generated spiritual content that constitutes sensitive personal data. The absence of encryption at rest for PII, flagged as an unremediated Batch 1 gap [3], extends to this devotional content. Additionally, the devotional generation endpoint and reading plan progress data reveal personal spiritual practices that warrant field-level encryption and strict access controls beyond what the current architecture provides.

5 Pillar 3: Reliability

Third-Party Bible API Resilience. The ESV API’s 5,000 requests/day limit [1] represents a hard ceiling that, without proper management, could leave users without scripture text during peak usage periods such as Sunday morn-

ings or Advent/Lent seasons. The current architecture lacks the circuit breaker pattern recommended by AWS Prescriptive Guidance, which tracks failure states (closed → open → half-open) and prevents cascading failures when a downstream dependency becomes unhealthy [2]. AWS Powertools for Lambda now includes a built-in circuit breaker utility specifically designed for this purpose [15]. The AWS Well-Architected Reliability Pillar (REL05-BP03) prescribes exponential backoff with jitter and capped retry limits [16], while REL05-BP01 recommends transforming hard dependencies into soft ones by returning cached values or predetermined static responses when a dependency fails [17]. The local KJV SQLite fallback [1] partially addresses this, but the degradation path is not formalized through circuit breaker state management.

Easter Computus Algorithm Correctness. The Anonymous Gregorian Method implemented in Flow 8 produces correct Easter dates for the years 2000–2100 [1], but the algorithm incorporates three century-based correction factors — the solar equation, the lunar equation, and the starting-point correction — all of which shift values at century boundaries such as year 2100 [18]. Academic research has noted persistent doubts about the reliability of some Easter algorithm results, with Gregorian Easter algorithms having a history spanning over two centuries with obscure edge cases [19]. While the year 2100 risk is distant, the absence of a validation test suite means that any refactoring or porting of the algorithm could introduce errors that silently propagate incorrect dates for all moveable feasts and, by extension, the entire liturgical calendar.

Offline Sync and Data Integrity. The current offline sync strategy uses last-write-wins conflict resolution [5], which is inadequate for bookmark and annotation data where concurrent edits from multiple devices could silently overwrite user content. AWS AppSync supports three superior conflict resolution strategies: Optimistic Concurrency (which rejects conflicting mutations and prompts client retry with the updated version), Automerge (field-level merge), and custom Lambda resolvers for application-specific logic [7]. AWS Amplify DataStore provides automatic versioning, conflict detection, and delta synchronization when devices transition from offline to online [6, 20].

Streak Integrity. The reading plan progress tracking with its 48-hour grace period [5] is vulnerable to race conditions when users complete readings on multiple devices simultaneously. Without conditional writes, two devices could both read the same streak count, independently increment it, and write back — resulting in a lost update. DynamoDB’s conditional writes with `ConditionExpression` checking the last-completion timestamp, combined with strongly consistent reads, would prevent these race conditions while providing 99.99% availability with automatic three-AZ replication [8, 9].

Absent Dead Letter Queues. The DLQ gap identified as P0-Critical in Batch 1 [3] extends to Flow 8’s scripture delivery and notification queues. Failed streak reminder notifications (sent at the 44-hour mark via AWS SNS [5]) and failed devotional generation requests would be silently lost without DLQs, with no mechanism for retry or operational alerting.

6 Pillar 4: Performance Efficiency

Full-Text Search Scalability. PostgreSQL tsvector with GIN indexing provides the foundation for scripture search across all 7 translations [1], but the current implementation faces scaling risks. According to PostgreSQL documentation, GIN indexes are the preferred type for full-text search with performance scaling logarithmically with unique words [21]. However, key optimizations are missing: dedicated tsvector columns with triggers to pre-compute and cache lexemes rather than computing them at query time [10], and tuned `maintenance_work_mem` settings during index operations [22]. As the corpus grows with additional translations and user-generated annotations, query performance will degrade without these optimizations. The sub-300ms p95 target [1] is achievable at current scale but requires proactive index management to maintain.

Audio Streaming Latency. The two-tier audio architecture achieves less than 1.5 seconds first-byte latency on LTE for pre-recorded HLS streams served via CloudFront [5], which is acceptable. However, the Polly Neural fallback path for NRSV, NLT, NAB, and MSG translations introduces variable latency when audio must be synthesized on demand. Amazon Polly supports asynchronous synthesis via `StartSpeechSynthesisTask` for large text passages, storing output directly to S3 [23], but the current architecture does not appear to pre-generate audio for commonly accessed passages. CloudFront cache hit ratio optimization is critical here — poor cache hit ratios increase both latency and origin load [24].

Caching Strategy Effectiveness. The Redis 24-hour TTL for scripture verses [1] is well-calibrated for content that changes infrequently, and the differentiated HTTP caching strategy (3600s for calendar, 86400s for cycle year, ETag for sermon series [13]) demonstrates thoughtful cache design. The 50MB mobile offline budget with its allocation of approximately 16.8MB for audio and 33MB for text and metadata [5] is tight but workable, though the 90% alert threshold [5] provides limited headroom for users who bookmark extensively or access multiple translations.

7 Pillar 5: Cost Optimization

AWS Polly Cost Exposure. Polly Neural voices are priced at \$16.00 per million characters [11], and on-demand synthesis for every scripture audio request would generate significant costs at scale. The recommended approach is to pre-generate all commonly accessed chapters using Standard voices (\$4.00 per million characters) or Neural voices for quality-critical passages, store the resulting MP3 files in S3 at approximately \$0.023/GB/month, and serve them via CloudFront [11]. AWS explicitly permits caching and replaying generated speech at no additional cost [11], making pre-generation the clear cost-optimal strategy. The current 30-day S3 TTL for Polly-generated audio [5] should be extended to permanent storage for pre-generated content.

Redis to Valkey Migration. AWS has documented that upgrading ElastiCache clusters from Redis OSS to Valkey can achieve up to 60% cost savings [12]. For Flow 8's scripture text caching workload — which involves relatively simple key-value lookups with TTL-based eviction — Valkey provides full compatibility with significant cost reduction. ElastiCache Serverless with Valkey compatibility would further optimize costs by eliminating the need to provision fixed cache node capacity.

External API Call Reduction. The ESV API's 5,000 requests/day limit [1] creates both a reliability constraint and a cost consideration (overage charges or service suspension). The Redis caching layer with 24-hour TTL [1] significantly reduces external calls, but additional strategies include caching full chapter text rather than individual verses, pre-warming the cache for upcoming Sunday readings based on the lectionary calendar, and using the local KJV SQLite database as a zero-cost fallback for users who accept that translation.

8 Pillar 6: Sustainability

Compute Waste Reduction. The most impactful sustainability improvement for Flow 8 is eliminating repeated Polly synthesis calls through pre-generation. Each on-demand synthesis request consumes GPU compute resources that could be avoided entirely by generating audio once and serving it from S3/CloudFront. This aligns with the AWS Well-Architected Sustainability Pillar's emphasis on minimizing redundant computation. The Graviton2/3 migration for Lambda functions, identified as a P2 item in the Batch 3 roadmap [3], would reduce energy consumption for all Flow 8 compute workloads while also improving price-performance.

Data Transfer Optimization. The 7-day pre-cache strategy for mobile offline support [5] should be right-sized based on actual usage patterns. If analytics show that most users access only 2–3 days of content before reconnecting, reducing the pre-cache window would decrease redundant data transfer over cellular networks. The delta synchronization approach recommended by AWS Amplify DataStore [20] would further minimize data transfer by syncing only changed records rather than full datasets on reconnect. The Valkey migration [12] also contributes to sustainability by reducing the memory footprint of the caching layer, allowing smaller instance sizes with lower energy consumption.

9 Flow 8-Specific Risk Deep Dive

This section examines the six new risks unique to Flow 8 that were not present in Batches 1–3.

Bible API Rate Limiting and Availability. The ESV API's 5,000 requests/day cap [1] is a hard constraint that could be exhausted during peak usage — a large church with 2,000 members accessing Sunday readings simultaneously could consume a significant portion of the daily quota in minutes. The absence of a circuit breaker means that once the rate limit is hit, subsequent requests will fail without graceful degradation. The AWS Resilience Analysis Framework lists rate limiting, load shedding, exponential backoff with jitter, and circuit breakers as key mitigation strategies [25]. The recommended architecture is a three-tier fallback: Redis cache → Bible.api secondary → local KJV SQLite, with circuit breaker state management at each tier boundary.

Computus Century-Boundary Risk. While the Anonymous Gregorian Method is mathematically correct for 2000–2100, implementations that hard-code century-based corrections or use integer truncation incorrectly will produce wrong Easter dates starting in 2100 [18]. The Gregorian reform's epact-based corrections adjust the ecclesiastical moon cycle, and the Julian calendar accumulates an error of approximately one day every 128 years [26]. The mitigation is straightforward: implement a comprehensive validation test suite that verifies Easter dates against published tables for years spanning multiple century boundaries (e.g., 2099–2101, 2199–2201), and run these tests as part of the CI/CD pipeline.

Offline Sync Conflicts. The last-write-wins strategy [5] is acceptable for ephemeral cache data but creates data loss risk for user-generated content such as bookmarks, highlights, and annotations stored in `scripture_bookmarks` [1]. AWS AppSync's Optimistic Concurrency or Automerge strategies [7] would preserve user intent across concurrent offline edits. The additional complexity of managing offline writes, as noted in AWS Amplify documentation [27], is justified for this high-value user data.

Reading Plan Streak Integrity. The 48-hour grace period with streak reset [5] creates a user experience that is

highly sensitive to data consistency. A lost streak due to a race condition or sync failure would undermine user trust in the platform. DynamoDB conditional writes provide the necessary atomicity — checking that the last-completion timestamp falls within the grace window before incrementing the streak count [8] — while the service’ s 99.99% availability SLA ensures the streak tracking layer itself does not become a reliability bottleneck [9].

Audio Streaming Latency for Polly Fallback. On-demand Polly synthesis for translations without pre-recorded audio (NRSV, NLT, NAB, MSG) introduces unpredictable latency that degrades the user experience compared to the sub-1.5-second first-byte target achieved by HLS streams [5]. Amazon Polly’ s bidirectional streaming over HTTP/ 2 offers lower latency for real-time synthesis [28], but the optimal solution remains pre-generation of audio for all commonly accessed passages, stored in S3 and served via CloudFront [23, 24].

Search Indexing at Scale. The tsvector search implementation [1] requires dedicated columns with triggers to maintain pre-computed lexemes [10] rather than computing them at query time. Without this optimization, search latency will increase linearly with corpus size as new translations or user annotations are added. GIN index maintenance also requires careful tuning — for bulk data loads, dropping and recreating the index is more efficient than incremental updates [22].

10 Cross-Batch Consolidated Findings (Batches 1-4)

With the completion of Batch 4, all eight ROCA flows have been assessed. The cumulative findings reveal a pattern of compounding architectural risk: six systemic gaps identified in Batch 1 have propagated unremediated through every subsequent flow, with each batch adding flow-specific risks on top of the foundational deficiencies [3, 4].

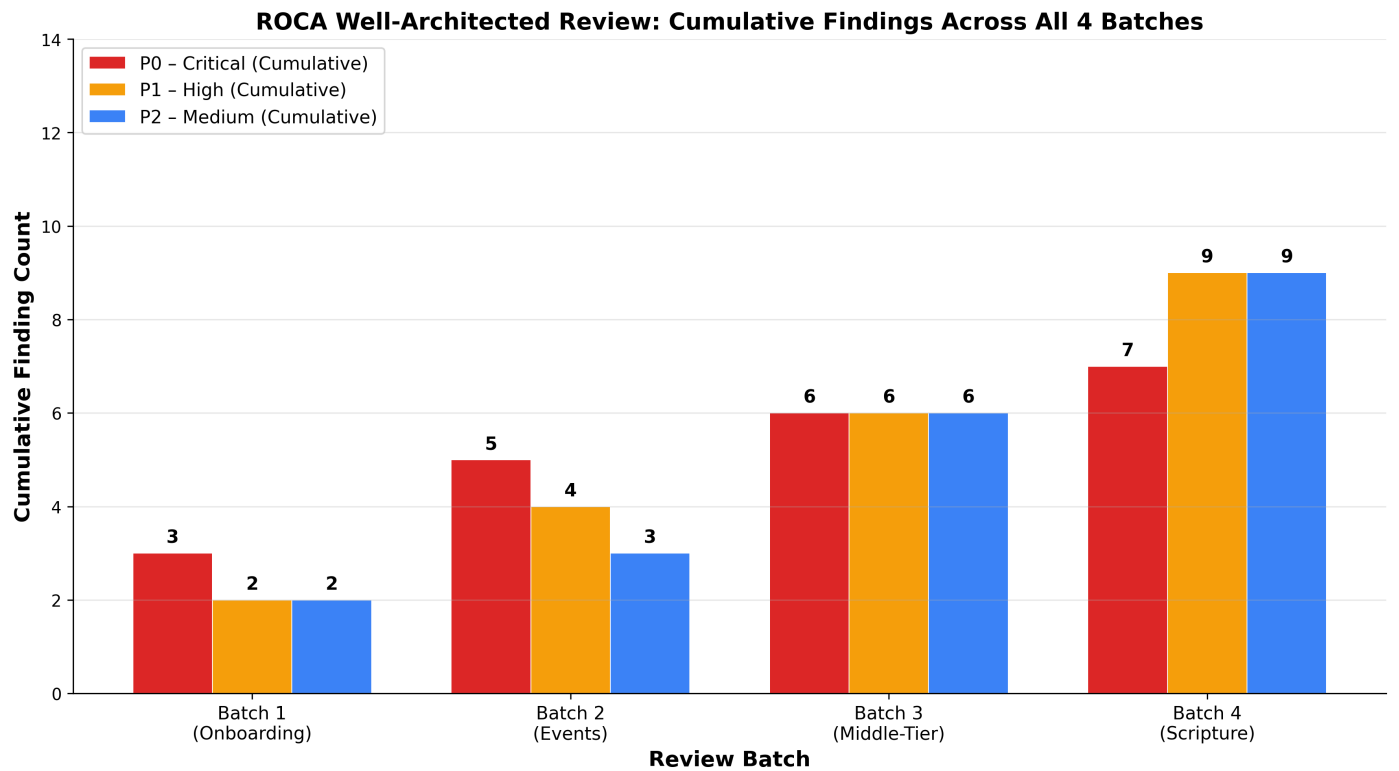


Figure 1: Cumulative Findings Across All 4 Batches

Chart Explanation: This chart shows the cumulative growth of findings across all four review batches. P0-Critical findings have grown from 3 in Batch 1 to 7 across all batches, P1-High from 2 to 9, and P2-Medium from 2 to 9, for a total of 25 distinct findings. The steady accumulation reflects both the discovery of new flow-specific risks and the persistent unremediated state of foundational gaps [3, 4, 14].

The following table consolidates the 12 most critical findings from Batches 1-3 alongside the 6 new Flow 8-specific risks:

#	Finding	Severity	Origin Batch	Effort	Pillar(s)
1	Absent DLQs on all SQS queues	P0-Critical	Batch 1	~3 days	Reliability [3]

#	Finding	Severity	Origin Batch	Effort	Pillar(s)
2	Missing WAF + GuardDuty	P0-Critical	Batch 1	~5 days	Security [4]
3	JWT rotation absent	P0-Critical	Batch 1	~4 days	Security [3]
4	Cancellation saga missing	P0-Critical	Batch 2	~10 days	Reliability [3]
5	SES bounce + FCM v1 migration	P0-Critical	Batch 3	~5 days	Reliability [3]
6	PCI DSS v4.0 compliance gaps	P0-Critical	Batch 2	~5 days	Security [14]
7	Bible API circuit breaker absent	P0-Critical	Batch 4	~3 days	Reliability [2]
8	Embedded state machines (all flows)	P1-High	Batch 1	~15 days	Ops Excellence [3]
9	RSVP race conditions	P1-High	Batch 2	~8 days	Reliability [4]
10	No EventBridge event bus	P1-High	Batch 1	~10 days	Ops Excellence [3]
11	Streak integrity (concurrent writes)	P1-High	Batch 4	~4 days	Reliability [8]
12	Offline sync (last-write-wins)	P1-High	Batch 4	~5 days	Reliability [7]
13	tsvector optimization needed	P1-High	Batch 4	~3 days	Performance [10]
14	Polly audio pre-generation	P2-Medium	Batch 4	~3 days	Cost, Sustainability [11]
15	Valkey migration	P2-Medium	Batch 4	~3 days	Cost [12]
16	Computus validation suite	P2-Medium	Batch 4	~2 days	Reliability [18]

Table: Consolidated Critical Findings Across All 4 ROCA Review Batches

The six systemic gaps that propagate across every ROCA flow are: embedded state machines in application code instead of Step Functions, absent DLQs causing silent data loss, missing WAF/GuardDuty perimeter security, JWT authentication without rotation or PKCE, no EventBridge for event-driven decoupling, and no X-Ray distributed tracing [3]. The compounding pattern is clear — each foundational gap identified in Batch 1 amplifies risk in every subsequent flow, with WAF/GuardDuty and JWT gaps escalating from High to Critical once payment processing entered scope in Batch 2 [4].

11 Prioritized Remediation Roadmap

The consolidated remediation roadmap spans 90 days and approximately 95–115 engineering days of effort, incorporating both the Batch 1–3 backlog (approximately 77–95 days [3]) and the new Flow 8-specific items (approximately 18–20 additional days).

Chart Explanation: This Gantt chart visualizes the sequencing of all remediation work across three priority tiers. P0-Critical items (red) occupy Days 1–30 and address the most severe security and reliability gaps. P1-High items (amber) span Days 31–60 and focus on architectural modernization and data integrity. P2-Medium items (blue) fill Days 61–90 with cost optimization, sustainability improvements, and forward-looking validation [3].

P0 — Critical (Days 1–30, approximately 30 engineering days). The first phase addresses items that pose immediate security or data-loss risk. Bible API circuit breaker implementation using AWS Powertools [15] should be the first item, as it directly protects the user-facing scripture delivery path. DLQ deployment across all SQS queues (~3 days), WAF and GuardDuty activation (~5 days), JWT rotation via Cognito (~4 days), SES bounce and complaint handling with FCM HTTP v1 migration (~5 days), and the Event Cancellation saga via Step Functions (~10

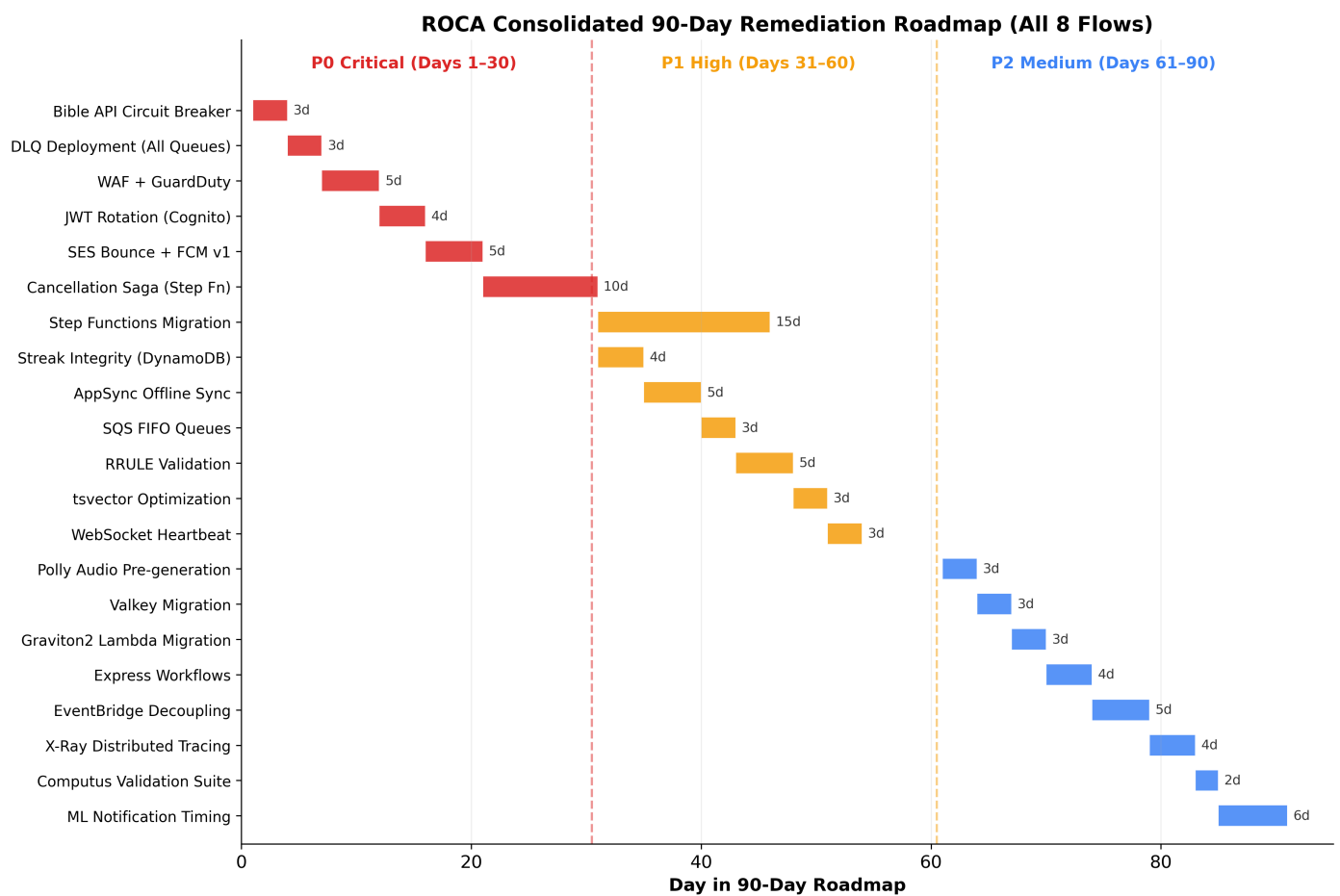


Figure 2: 90-Day Remediation Roadmap

days) complete the critical tier [3].

P1 — High (Days 31–60, approximately 38 engineering days). The second phase modernizes the architecture. Step Functions migration for all embedded state machines (~15 days) is the largest single item and delivers the highest operational excellence improvement [3]. Streak integrity via DynamoDB conditional writes (~4 days) [8], AppSync offline sync upgrade (~5 days) [6], SQS FIFO queue deployment (~3 days), RRULE input validation (~5 days), tsvector column optimization with triggers (~3 days) [10], and WebSocket heartbeat fixes (~3 days) round out this phase [3].

P2 — Medium (Days 61–90, approximately 30 engineering days). The final phase focuses on cost optimization and sustainability. Polly audio pre-generation to S3 (~3 days) [11], Valkey migration (~3 days) [12], Graviton2 Lambda migration (~3 days), Express Workflows (~4 days), EventBridge decoupling (~5 days), X-Ray distributed tracing (~4 days), Computus validation suite (~2 days) [18], and ML notification timing engine (~6 days) complete the roadmap [3].

The total effort of approximately 95–115 engineering days across 90 calendar days assumes a team of 2–3 engineers working in parallel, with P0 items requiring dedicated focus from the most senior team members due to their security and reliability implications. Organizations should treat the P0 tier as non-negotiable — the combination of absent WAF, missing JWT rotation, and no DLQs represents a security and reliability posture that falls below the minimum acceptable threshold for a production system handling personal spiritual data.

12 References

1. ROCA_Scripture_Lictionary_Design_v1.0.docx
2. <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>
3. AWS Well-Architected Framework Review for ROCA Middle-Tier Flows: Orchestration, Scheduling, and Notification Architecture
4. AWS Well-Architected Framework Review for ROCA Event Management Flows with Batch 1 Gap Analysis
5. ROCA_Scripture_Lictionary_Mobile_Screens_Batch2.docx
6. <https://docs.aws.amazon.com/whitepapers/latest/amplify-datastore-implementation/introduction.html>
7. https://docs.aws.amazon.com/en_en/appsync/latest/devguide/conflict-detection-and-resolution.html
8. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html>
9. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/disaster-recovery-resiliency.html>
10. <https://thoughtbot.com/blog/optimizing-full-text-search-with-postgres-tsvector-columns-and-triggers>
11. <https://aws.amazon.com/polly/pricing>
12. <https://aws.amazon.com/blogs/database/reduce-your-amazon-elasticache-costs-by-up-to-60-with-valkey-and-cudos/>
13. ROCA_Scripture_Lictionary_Web_Screens_Batch1.docx
14. AWS Well-Architected Framework Review for ROCA Event Management Flows: Payment Security, Concurrency, and Operational Resilience Assessment
15. https://docs.aws.amazon.com/powertools/python/latest/utilities/circuit_breaker/
16. https://docs.aws.amazon.com/wellarchitected/2024-06-27/framework/rel_mitigate_interaction_failure_limit_retries
17. https://docs.aws.amazon.com/wellarchitected/2022-03-31/framework/rel_mitigate_interaction_failure_graceful_deg
18. <https://www.sqlservercentral.com/scripts/calculating-easter-in-sql>
19. <https://link.springer.com/article/10.1007/s00407-004-0078-5>
20. <https://docs.aws.amazon.com/whitepapers/latest/amplify-datastore-implementation/amplify-datastore-overview.html>
21. <https://www.postgresql.org/docs/9.4/static/textsearch-indexes.html>
22. <https://www.postgresql.org/docs/14/gin-tips.html>
23. <https://docs.aws.amazon.com/polly/latest/dg/asynchronous.html>

24. <https://docs.aws.amazon.com/whitepapers/latest/amazon-cloudfront-media/cloudfront-configuration-best-practices.html>
25. [https:// docs.aws.amazon.com/ prescriptive-guidance/ latest/ resilience-analysis-framework/ mitigation-strategies.html](https://docs.aws.amazon.com/prescriptive-guidance/latest/resilience-analysis-framework/mitigation-strategies.html)
26. https://www.academia.edu/110284866/Christian_lunisolar_calendar
27. <https://docs.amplify.aws/gen1/android/build-a-backend/more-features/datastore/migrate-from-datastore/offline-first/>
28. [https:// aws.amazon.com/ blogs/ machine-learning/ introducing-amazon-polly-bidirectional-streaming-real-time-speech-synthesis-for-conversational-ai](https://aws.amazon.com/blogs/machine-learning/introducing-amazon-polly-bidirectional-streaming-real-time-speech-synthesis-for-conversational-ai)